BENCHMARK DE RUNTIMES | 2026

Comparativo de runtimes para fluxos assíncronos

Aprovação, enriquecimento e sincronização de catálogo

30	100	0	3
CENÁRIOS	VUs NO PICO	ERROS FUNCIONAIS	RUNTIMES

Leitura executiva

Nas implementações sintéticas e sob os limites iguais desta execução, Go apresentou a maior taxa média de jobs concluídos nos três fluxos: 24,92 jobs/s, contra 20,84 em Node.js + NestJS e 19,32 em .NET. A duração média dos job p95 em Go também foi menor. Esses números ajudam a direcionar uma próxima rodada; não medem a Indexa inteira em produção.

Runtime	Jobs/s médio nos 3 fluxos	Média dos job p95	RAM máxima média
Node.js + NestJS	20,84	1,055 s	114,2 MiB
.NET 10	19,32	0,933 s	103,1 MiB
Go	24,92	0,346 s	21,7 MiB

Médias simples das três execuções stress por runtime. A média dos job p95 não é um percentil agregado; RAM é a média dos picos medidos por cenário.

Escopo: serviços equivalentes criados para este benchmark, fora dos repositórios da Indexa. Adapters de publicação, origem do catálogo e provedor de IA foram simulados.

Matriz executada em 13/10/2026 UTC | Relatório preparado em 14/10/2026

01 | Escopo e metodologia

O benchmark reproduz o formato assíncrono dos fluxos principais: o cliente cria um job; o serviço persiste o pedido e sua outbox; um dispatcher publica na Redis Streams; workers processam os itens e registram o resultado no PostgreSQL.

Fluxos modelados

Fluxo	Operação representada	Fronteira simulada
Aprovação	Submeter itens e persistir o resultado por anúncio.	Adapter de publicação determinístico.
Enriquecimento	Gerar e persistir título, slug, descrição e atributos.	Transformação local; sem modelo de IA.
Sincronização	Importar páginas sintéticas e atualizar o catálogo.	Fixture local; sem API de canal ou loja.

Carga e regras comuns

Configuração	Valor aplicado
Itens por job	100 em cada runtime.
Rampa stress	10 VUs por 10 s; 30 por 20 s; 60 por 20 s; 100 por 45 s; descida a zero em 10 s.
Carga mista quick	20 VUs por 60 s, 5 ms de latência simulada, alternando os três fluxos.
Recursos por serviço	2 vCPU e 2 GiB; PostgreSQL com os mesmos limites; Redis com 1 vCPU e 512 MiB.
Concorrência	8 jobs, 8 itens por job, página de 100 e pool PostgreSQL de 20 conexões.

Composição dos 30 cenários

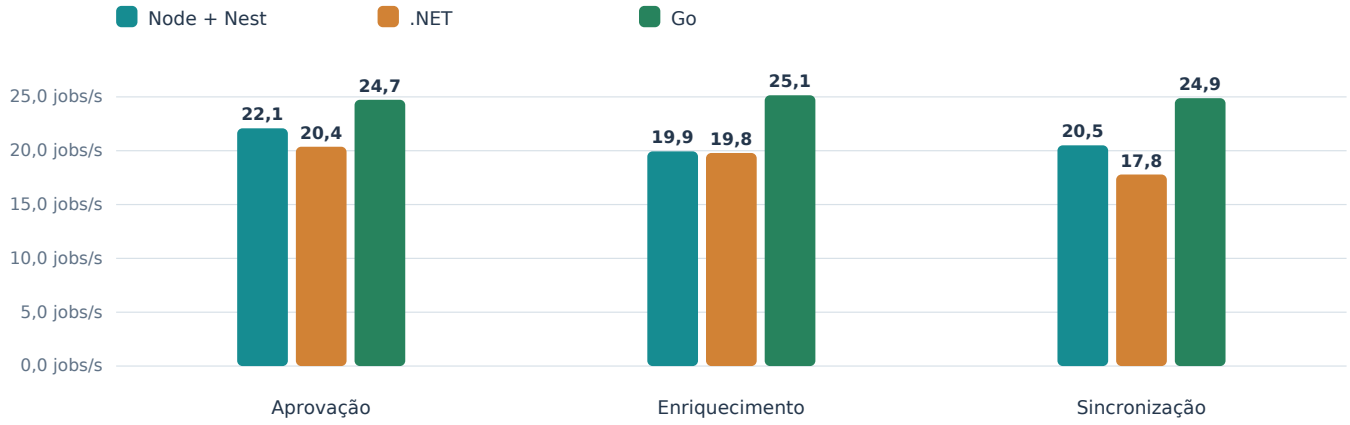
Para cada runtime: três smoke sem latência ou falha; os mesmos três com 5 ms simulados e falha esperada a cada 10 itens; três rampas stress sem latência ou falha; e uma carga mista quick. São 10 cenários por runtime. O perfil soak de 180 s existe no harness, mas não foi executado nesta matriz.

Validação: checks verificaram aceite HTTP, estado terminal, contagem de itens e falhas injetadas. Os 30 cenários passaram thresholds e registraram zero erros funcionais inesperados.

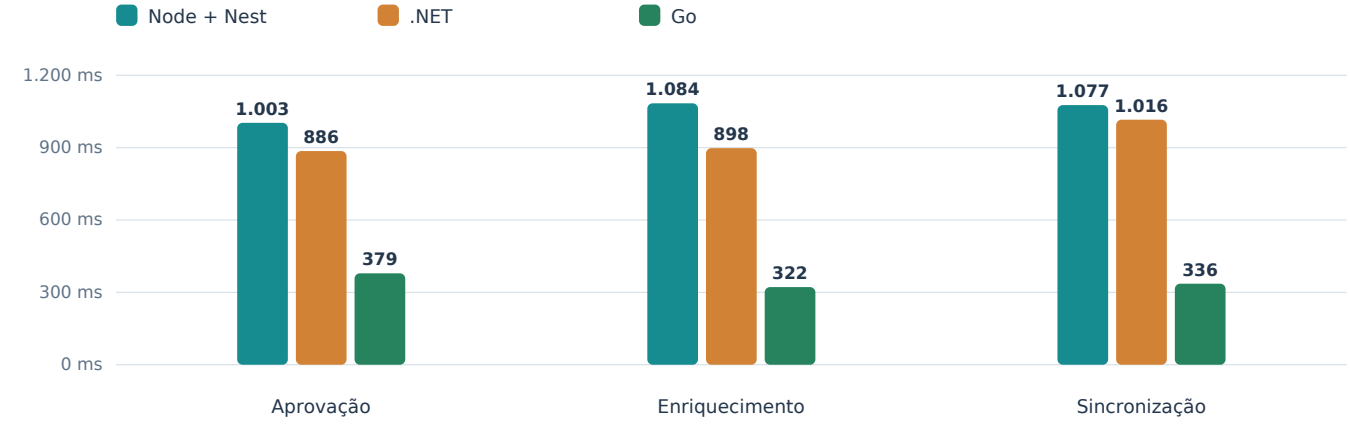
02 | Resultados de stress

Cada ponto compara o mesmo fluxo, 100 itens por job, sem latência externa e sem falhas injetadas. A carga sobe até 100 VUs e mantém esse nível por 45 segundos.

Jobs concluídos por segundo



Duração do job no p95



Go: +19,6% jobs/s sobre Node.js + NestJS e +29,0% sobre .NET. O job p95 médio ficou 67,2% abaixo de Node.js + NestJS e 63,0% abaixo de .NET. Resultado válido para este workload e esta máquina.

Jobs/s é a taxa de jobs que chegaram ao estado terminal. Job p95 é a duração medida pelo próprio serviço. E2E p95 inclui enqueue e polling; veja a tabela detalhada.

03 | Resultados por fluxo

CPU e memória são máximos observados nas amostras de docker stats durante cada cenário. A CPU pode ultrapassar 100%, indicando mais de um core ocupado.

Rampa stress por fluxo

Runtime	Fluxo	Jobs/s	Job p95	E2E p95	CPU máx.	RAM máx.
Node + Nest	Aprovação	22,08	1.003 ms	4.384 ms	46,3%	110,4 MiB
Node + Nest	Enriquecimento	19,94	1.084 ms	4.712 ms	38,8%	113,0 MiB
Node + Nest	Sincronização	20,50	1.077 ms	4.689 ms	37,1%	119,3 MiB
.NET	Aprovação	20,37	886 ms	4.888 ms	76,1%	96,1 MiB
.NET	Enriquecimento	19,79	898 ms	4.893 ms	93,4%	98,1 MiB
.NET	Sincronização	17,79	1.016 ms	5.417 ms	66,9%	115,2 MiB
Go	Aprovação	24,73	379 ms	3.590 ms	102,7%	21,4 MiB
Go	Enriquecimento	25,15	322 ms	3.796 ms	100,1%	21,5 MiB
Go	Sincronização	24,88	336 ms	3.999 ms	102,7%	22,1 MiB

Carga mista quick

Runtime	Jobs/s	E2E p95	CPU máx.	RAM máx.
Node.js + NestJS	21,08	1.188 ms	38,4%	119,6 MiB
.NET 10	18,99	1.397 ms	89,8%	107,5 MiB
Go	25,91	884 ms	99,6%	19,7 MiB

Perfil: 20 VUs por 60 s, com 5 ms simulados. Go concluiu 25,91 jobs/s, Node.js + NestJS 21,08 e .NET 18,99. E2E p95 inclui o tempo entre criar o job e observar seu estado final.

Os cenários com falha determinística eram smoke: falhavam a cada 10 itens e validavam contagens/status esperados. Essas falhas simuladas não são classificadas como erros inesperados.

04 | Interpretação e próximos passos

O que os números sustentam

Nesta matriz, Go sustentou mais jobs por segundo nos três fluxos, menor duração de job p95 e menor pico médio de memória. Node.js + NestJS e .NET também concluíram a carga com os checks esperados. Como adapters e transformações são sintéticos, o resultado orienta uma validação mais próxima da produção, não uma decisão isolada de migração.

Limites da comparação

Fronteira	Efeito na leitura
Adapters e IA simulados	Não mede latência, rate limits, custo ou variabilidade de canais, lojas e provedores de IA reais.
Redis Streams comum	Mantém a fila comparável, mas não compara BullMQ nem substitui o outbox/worker completo da Indexa.
Ambiente local	Os números dependem do host e dos limites de container registrados; não representam produção.
Sem soak nesta matriz	Não avalia estabilidade por horas, vazamentos de memória ou degradação prolongada.

Próxima rodada sugerida

Repetir os testes de stress para medir variação; rodar soak prolongado; registrar CPU, disco e versão do host; e substituir os stubs por fixtures representativas dos adapters e payloads reais, mantendo serviços externos fora da carga.

Ambiente registrado

Node.js v24.21.0 com NestJS 11.2.7; .NET 10.0.12; Go 1.27.2; k6 1.8.2; Docker 29.8.1. PostgreSQL e Redis foram compartilhados. A matriz começou às 20:20:49 UTC e terminou às 20:40:37 UTC em 13/10/2026.

Reprodução e artefatos

A implementação, o escopo, os summaries e os arquivos brutos comprimidos ficam em /home/kmas/Documentos/indexa-benchmark. Consulte README.md para comandos, dependências, limpeza dos containers e descompressão dos resultados.

Fontes: results/summary.md, results/matrix-environment.txt e BENCHMARK-SCOPE.md. Os arquivos JSON brutos do k6 foram preservados em results como .raw.json.gz.